

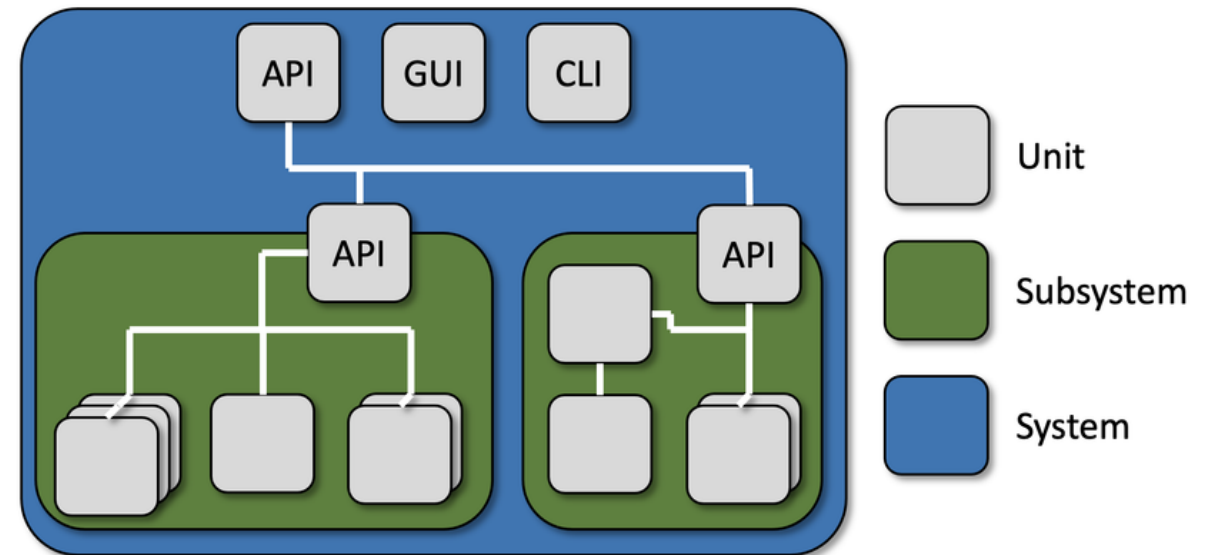
Lecture 6

Software Verification and Validation

Unit Testing and Test Automation

Testing Stages

- We interact with systems through **interfaces**.
 - APIs, GUIs, CLIs
- Systems built from **subsystems**.
 - With their own interfaces.
- Subsystems built from **units**.
 - Communication via method calls.
 - Set of methods is an interface.



Unit Testing

- Testing the smallest “unit” that can be tested.
 - Often, a class and its methods.
- Tested in isolation from all other units.
 - Mock the results from other classes.
- Test input = method calls.
- Test oracle = assertions on output/class variables.

Unit Testing

For a unit, tests should:

- Test all “jobs” associated with the unit.
 - Individual methods belonging to a class.
 - Sequences of methods that can interact.
- Set and check class variables.
 - Examine how variables change after method calls.
 - Put the variables into all possible states (types of values).

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

Unit Testing - Account

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

Unit tests should cover:

- Set and check class variables.
 - Can any methods change name, person number, balance?
 - Does changing those create problems?
- Each “job” performed by the class.
 - Single methods or method sequences.
 - Change the order methods are called.
 - Each outcome of each “job” (error handling, return conditions).

Unit Testing - Account

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

Some tests we might want to write:

- Execute constructor, verify fields.
- Check the name, change the name, make sure changed name is in place.
- Check that person number is correct.
- Check the balance, withdraw money, verify that new balance is correct.
- Check the balance, deposit money, verify that new balance is correct.

Unit Testing - Account

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

Some potential error cases:

- Withdraw more than is in balance.
- Withdraw a negative amount.
- Deposit a negative amount.
- Withdraw/Deposit a small amount
(potential rounding error)
- Change name to a null reference.
- Can we set an “malformed” name?
 - (i.e., are there any rules on a valid name?)

- Unit tests are used to test a single unit in isolation
 - Verifying that it works as expected
 - No matter the rest of the program would do
- Possible advantages ?
 - (Possibly) No inheritance of bugs or mistakes from made elsewhere
 - Narrow down on the actual problem
- Is it enough ?
 - No, by itself, but...
 - ... it is the foundation upon which everything is based!

Example Scenario

Please, imagine that you have to test a building

- Test if it has been constructed properly
- Test if it is able to resist to earthquake
-
- What types of “testing” will you do?

1. Unit Testing – *Testing individual parts (bricks, windows, pipes)*

You check:

- Are the **bricks** solid and made from the right material?
- Do **individual windows** open/close properly?
- Are **pipes** leak-free on their own?

Software analogy: Testing a single function or class in isolation.

2. Integration Testing – *Testing if parts work together*

You check:

- Does the **plumbing system** (pipes + faucets + drainage) work as a whole?
- Does the **elevator** (motor + control system + door) run smoothly?
- Does the **security system** connect the alarm, sensors, and control panel correctly?
- *Software analogy: Do different modules communicate and work together properly?*

Automated Unit Testing Tools

Many unit testing solutions exist for a wide range of languages and environments. Some common resources are:

- **JUnit**– is an open-source unit testing framework written in Java that has seen widespread use. It offers a structure for making automated tests, which can be performed repeatedly, as well as for reporting and verifying the results of such tests.
- **NUnit**– NUnit is a unit testing framework for C# and VB.NET, as well as other.NET languages. Comparable features to those of JUnit are provided.
- **x.Unit.net**– Inspired by JUnit and NUnit, xUnit.net is a unit testing framework for .NET languages. It's a great place to start when creating automated tests since it's easy to learn and adapt.

Automated Unit Testing Tools

- **PyTest**— PyTest is an automated testing, integration testing, and functional testing framework written in Python. It offers a straightforward environment in which to create and execute tests.
- **PHPUnit**— is a framework for developing and running unit tests in PHP. It supports test-driven development (TDD) and behavior-driven development (BDD), among other capabilities useful for creating and executing automated tests (BDD).
- **Jasmine**— a behavior-driven development (BDD) framework for JavaScript has you covered when it comes to developing and executing automated tests. It may be used to test applications on both the client and server.
- **Selenium**— is an open-source framework for testing web applications that includes support for browser automation. It's helpful for automated integration testing, functional testing, and unit testing.

Benefits of Unit Test Automation

- **Quality**— is increased because flaws and mistakes in the code are found earlier in the development process when they are less difficult and costly to rectify. Software quality and reliability may be increased if developers take the time to test and verify that their modules are functioning as expected.
- **Feedback**— on the quality and validity of code changes is provided fast through automated testing, enabling developers to swiftly detect and repair faults and mistakes.
- **Reduce risk**— With unit testing, you can be certain that each piece of code is being checked to make sure it's working as it should.
- **Increased efficiency**— Automation of the unit testing process increases productivity since it requires less time and resources than traditional manual testing.
- **Improved collaboration**— A uniform testing framework and the assurance that code changes are completely verified before being merged into the system are two ways automated testing may improve cooperation among developers.

Executing Tests

How do you run test cases on the program?

- System level: could run code and check results by hand.
- Please don't do this.
 - Humans are slow, expensive, and error-prone.
 - Exception - exploratory and acceptance testing.
- Test design requires effort and creativity.
- Test execution should not.

Test Automation

- Development of software to separate repetitive tasks from creative aspects of testing.
- Control over how and when tests are executed.
 - Control environment and preconditions/setup.
 - Automatic comparison of predicted and actual output.
 - Automatic hands-free re-execution of tests.

What is Test Scaffolding?

- Test scaffolding refers to the **supporting code and structures** used to test a software component in isolation.
- It creates a controlled environment where the unit under test can run without relying on real external dependencies.

Why Do We Need Test Scaffolding?

- To isolate the unit under test
- To avoid dependency on external systems (DB, APIs, services)
- To make tests faster and more reliable
- To enable testing before full system integration

Test Scaffolding

- Test scaffolding is a set of programs written to support test automation.
 - Not part of the product, often temporary
- Allows for:
 - Testing before all components complete.
 - Testing independent components.
 - Control over testing environment.

Test Scaffolding

Key Components of Test Scaffolding:

- **Mocks:** Simulated objects that mimic real behavior
- **Stubs:** Return predefined responses
- **Drivers:** Execute the test cases
- **Test Data Generators:** Provide input data
- **Setup/Teardown:** Prepare and clean test environments

Example Scenario

You are testing a `PaymentService` class.

Dependencies:

- Database
- External payment API

Problem:

- Real dependencies are slow, costly, or unavailable

Solution:

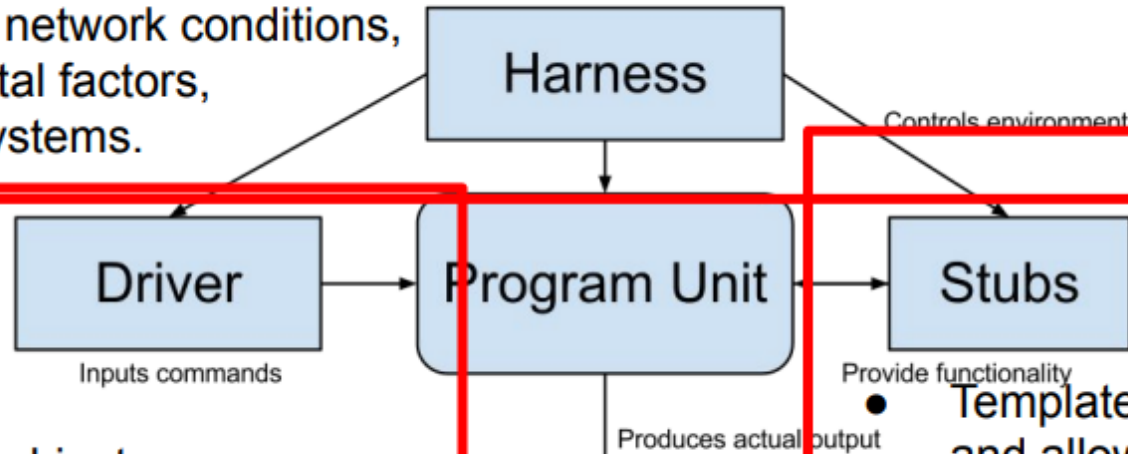
- Replace them with **mocks and stubs**

Instead of calling a real API:

- Mock API returns: "Payment Successful"
- Stub database returns predefined user data
- This allows testing only the business logic.

Test Scaffolding

- Simulates the execution environment.
- Can control network conditions, environmental factors, operating systems.



- Initializes objects
- Initializes parameter variables
- Performs the test
- Performs any necessary cleanup steps.

- Templates that provide functionality and allow testing in isolation

- Checks the correspondence between the produced and expected output and renders a test verdict.

Writing an Executable Test Case

- Test Input
 - Any required input data.
- Expected Output (Test Oracle)
 - What should happen, i.e., values or exceptions.
- Initialization
 - Any steps that must be taken before test execution.
- Test Steps
 - Interactions (e.g., method calls), and output comparisons.
- Tear Down
 - Steps that must be taken after execution to prepare for the next test.

Writing a Unit Test

JUnit is a Java-based toolkit for writing executable tests.

- Choose a target from the code base.
- Write a “testing class” containing a series of unit tests centered around testing that target.

```
public class Calculator {  
    public int evaluate (String  
        expression) {  
        int sum = 0;  
        for (String summand:  
            expression.split("\\+"))  
            sum += Integer.valueOf(summand);  
        return sum;  
    }  
}
```

“Programming today is a race between software engineers striving to build bigger and better idiot-proof programs, and the Universe trying to produce bigger and better idiots. So far, the Universe is winning.”

— **Rick Cook, The Wizardry Compiled**

JUnit Idea

- JUnit promotes the idea of "**first testing then coding**", which emphasis on setting up the test data for a piece of code which can be tested first and then can be implemented.
- This approach is like "**test a little, code a little, test a little, code a little...**" which increases programmer productivity and stability of program code that reduces programmer stress and the time spent on debugging.

JUnit Test Skeleton

@Test annotation defines a single test:

@Test

Type of scenario, and expectation on outcome.

I.e., testEvaluate_GoodInput() or testEvaluate_NullInput()

```
public void test<Feature or Method Name>_<Testing Context>() {  
    //Define Inputs  
    try  
    { //Try to get output.  
    }  
    catch(Exception error)  
    {  
        fail("Why did it fail?");  
    }  
    //Compare expected and actual values through assertions or through  
    //if-statements/fail commands  
}
```

Writing JUnit Tests

```
public class Calculator {
```

```
    public
```

Each test is denoted with keyword **@test**.

```
        int sum = 0;
```

```
        for (String summand:
```

```
            expression.split
```

```
            sum += Integer.valueOf(summand);
```

```
        return sum;
```

```
    }
```

```
}
```

```
import static org.junit.Assert.assertEquals;
```

```
import org.junit.Test;
```

```
public class CalculatorTest {
```

```
    @Test
```

```
    void testEvaluate_Valid_ShouldPass(){
```

```
        Calculator calculator = new Calculator();
```

```
        int sum = calculator.evaluate("1+2+3");
```

```
        assertEquals(6, sum);
```

```
    }
```

```
}
```

Initialization

Test Steps

Oracle

Input

Writing JUnit Tests

@Test

```
public void testWithdraw_validAmount() {  
    Account acc = new Account("Ali", 1000);  
    acc.withdraw(200);  
    assertEquals(800, acc.getBalance());  
}
```

Test Fixtures - Shared Initialization

@BeforeEach annotation defines a common test initialization method:

```
@BeforeEach
public void setUp() throws Exception
{
    this.registration = new Registration();
    this.registration.setUser("userTest");
}
```

Test Fixtures - Teardown Method

- **@AfterEach** annotation defines a common test tear down method:

```
@AfterEach
```

```
public void tearDown() throws Exception
```

```
{
```

```
    this.registration.logout();
```

```
    this.registration = null;
```

```
}
```

More Test Fixtures

- **@BeforeAll** defines initialization to take place before any tests are run.
- **@AfterAll** defines tear down after all tests are done.

```
@BeforeAll
public static void setUpClass() {
    myManagedResource = new
        ManagedResource();
}

@AfterAll
public static void tearDownClass()
throws IOException {
    myManagedResource.close();
    myManagedResource = null;
}
```

Assertions

Assertions are a "language" of testing - constraints that you place on the output.

- assertEquals, assertEquals
- assertFalse, assertTrue
- assertNull, assertNotNull
- assertEquals, assertEquals

assertNotNull

- Test passes if Object is not null.

assertNull

- Test passes if Object is null.

assertEquals

- Asserts equality of two values

assertTrue

- Test passes if condition is True

assertFalse

- Test passes if condition is False

assertSame

- Test passes if the two Objects are the same Object

assertEquals

Compares two items for equality.

- For user-defined classes, relies on `.equals` method.

- Compare field-by-field

- `assertEquals(studentA.getName(), studentB.getName())`

rather than

`assertEquals(studentA, studentB)`

- `assertArrayEquals`

compares arrays of items.

```
@Test
public void testAssertEquals() {
    assertEquals("failure - strings are not
equal", "text", "text");
}
```

```
@Test
public void testAssertArrayEquals() {
    byte[] expected = "trial".getBytes();
    byte[] actual = "trial".getBytes();
    assertArrayEquals("failure - byte arrays
not same", expected, actual);
}
```

assertSame, assertNotSame

```
@Test
public void testAssertNotSame() {
    assertNotSame("should not be same Object",
studentA, new Object());
}
```

```
@Test
public void testAssertSame() {
    Student studentB = studentA;
    assertSame("should be same", studentA,
studentB);
}
```

- Checks whether two objects are clones.
- Are these variables aliases for the same object?
 - assertEquals uses **.equals()**
 - assertSame uses **==**

assertNull, assertNotNull

- Take in an object and checks whether it is null/not null.
- Can be used to help diagnose and void null pointer exceptions.

```
@Test
public void testAssertNotNull() {
    assertNotNull("should not be null",
        new Object());
}
```

```
@Test
public void testAssertNull() {
    assertNull("should be null", null);
}
```

Unit Testing - Account

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

Withdraw money, verify balance.

```
@Test
public void testWithdraw_normal() {
    // Setup
    Account account = new Account("Test McTest", "19850101-1001", 48.5);
    // Test Steps
    double toWithdraw = 16.0; //Input
    account.withdraw(toWithdraw);
    double actual = account.getBalance();
    double expectedBalance = 32.5; // Oracle
    assertEquals(expected, actual); // Oracle
}
```

Unit Testing - Account

Account
- name - personnummer - balance
Account (name, personnummer, Balance) withdraw (double amount) deposit (double amount) changeName(String name) getName() getPersonnummer() getBalance()

- Withdraw more than is in balance.
 - (should throw an exception with appropriate error message)

```
@Test
public void testWithdraw_moreThanBalance() {
    // Setup
    Account account = new Account("Test McTest", "19850101-1001", 48.5);
    // Test Steps
    double toWithdraw = 100.0; //Input
    Throwable exception = assertThrows(
        () -> { account.withdraw(toWithdraw); } );
    assertEquals("Amount 100.00 is greater than balance 48.50",
        exception.getMessage()); // Oracle
}
```

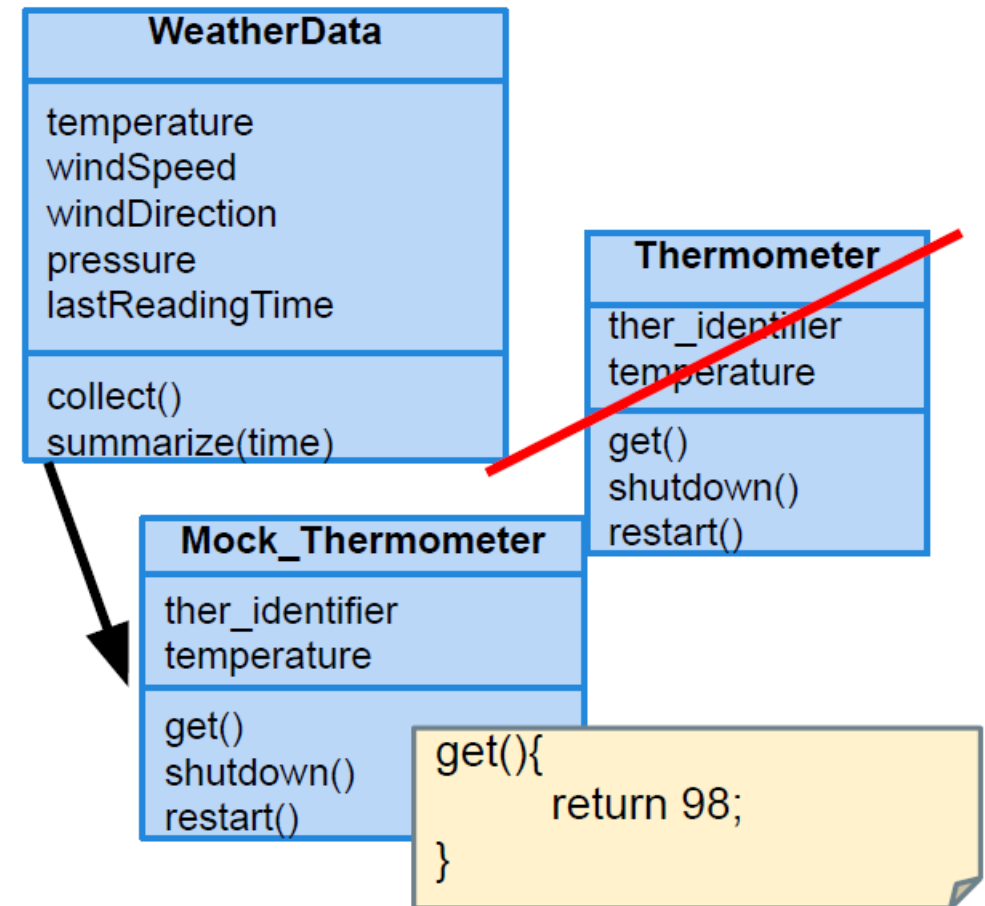
Scaffolding

- Mock objects and drivers are written as replacements for other parts of the system.
 - May be required if pieces of the system do not exist.
- Scaffolding allows control over test execution and greater observability to judge test results.
 - Simulate dependencies and test components in isolation.
 - Ability to set up specialized testing scenarios.
 - Ability to replace part of the program with a version more suited to testing.

Unit Testing - Object Mocking

Unit may depend on unfinished (or untested) components. Can **mock** those components.

- Same interface as real component, but hand-created simulation.
- Can be used to simulate abnormal operation or rare events.
 - Ex. Place exact data in database needed to hit special outcome.



Mocking Example

- Declare a mock object:

```
LinkedList mList = mock(LinkedList.class);
```

- Specify method behavior:

```
when(mList.get(0)).thenReturn("first");
```

- Returns "first": `mList.get(0);`
- Returns null: `mList.get(99);`
- Because behavior for "99" is not specified.

```
when(mList.get(anyInt())).thenReturn("element");
```

- `mList.get(0)`, `mList.get(99)` both return "element", as all input are specified.

Mocking Within a Test

@test

```
public void temperatureTest(){  
    Thermometer mockTherm = mock(Thermometer.class);  
    when(mockTherm.get()).thenReturn(98);  
    WeatherData wData = new WeatherData();  
    wData.collect(mockTherm);  
    assertEquals(98,wData.temperature);  
}
```

Mocking Within a Test

```
@Test
public void createUser() {
    // create the instance we'd like to test
    UserManager manager = new UserManagerImpl();
    // create the dependencies we'd like mocked
    Mock mailer = mock(Mailer.class);
    Mock dao = mock(UserDAO.class);
    // wire them up to our primary component, the user manager
    manager.setMailer((Mailer)mailer.proxy());
    manager.setDAO((UserDAO)dao.proxy());
    // specify expectations
    dao.saveUser() must return true;
    expect invocation dao.saveUser() with parameter "tester";
    dao.sendMail must return true;
    expect invocation dao.sendMail with parameter "tester"
    // invoke our method
    manager.createUser("tester");
    // verify that expectations have been met
    verifyExpectations();
}
```

Test runner

- Test runner is used for executing the test cases.
- Here is an example which assumes TestJunit test class already exists.

```
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {
    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(TestJunit.class);
        for (Failure failure : result.getFailures()) {
            System.out.println(failure.toString());
        }
        System.out.println(result.wasSuccessful());
    }
}
```

Assignment 2

Create New Account Page

First Name	
Last Name	
E-mail	
Password	
Date of Birth(dd/mm/yyyy)	
Confirm Password	
	<i>SUBMIT</i>

In this assignment, you're the quality engineer for the submit feature. Your job is to *break it* before the users do. Think like a hacker, a tester, and a software engineer. Validate every input, write tests for every case — and automate the testing so it runs itself!

Assignment 2 cont.

- You will make unit test on Create New Account Page.
- We worked on Junit examples but you can use any platform and tool you want.
- Use technics like Equivalence Partitioning, Boundary Value Analyses..
Create at least:15 test cases.
Use: Assertions , Setup & teardown methods.
- You will work in groups of 2 and make presentations at May 4th.
You will explain how you run the code and unit test on Create New Account Page.
- You will upload your report(code and explanation of your work) on classroom until May 5.

NOTE: Integrate your project with **GitHub Actions** to automate test execution.

Show:Commit history

Pull Requests (PRs)

Automated test results